



PYTHON

Seconed Session



Speaker. Majid Saqr



TOPICS COVERED

PYTHON BASICS

- Day 1: Introduction to Python and basic syntax, data types, and variables.
- Day 2: Exploring control structures (if statements, loops), functions, and modular programming.

OOP

- Day 3: Introduction to OOP concepts (classes, objects).
- Day 4: Understanding inheritance, polymorphism, and encapsulation.

TOPICS COVERED (P2)

DATA STRUCTURES AND ALGORITHMS

- Day 5: Introduction to fundamental data structures (lists, tuples, dictionaries).
- Day 6: Delving into algorithms and analysis, focusing on searching algorithms.

APPLICATIONS AND PROJECTS

- Day 7: Engage in a larger-scale programming project incorporating concepts learned throughout the course.
- Day 8: Present and discuss your project, showcasing your understanding and practical application of Python.



WHAT WE WILL TALK ABOUT ?

- 1) control structures (IF – IF else – IF elif else)
- 2) Loops (For – While)
- 3) Functions
- 4) modular programming.



Python Conditions and If statements

Python supports the usual logical conditions from mathematics:

Equals: $a == b$

Not Equals: $a != b$

Less than: $a < b$

Less than or equal to: $a \leq b$

Greater than: $a > b$

Greater than or equal to: $a \geq b$



If - Elif

The **elif** keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

Example :

```
a = 33
```

```
b = 33
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
elif a == b:
```

```
    print("a and b are equal")
```

Speaker. Majid Saqr



If – elif - else

The **else** keyword catches anything which isn't caught by the preceding conditions.

Example :

```
b = 200 , b = 33
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
elif a == b:
```

```
    print("a and b are equal")
```

```
else:
```

```
    print("a is greater than b")
```

Speaker. Majid Saqr



Short Hand

Short Hand If

Example : `if a > b: print("a is greater than b")`

Short Hand If ... Else

Example : `print("A") if a > b else print("B")`

You can also have multiple else statements on the same line:

Example : `print("A") if a > b else print("=") if a == b else print("B")`



Logical operator

And

The and keyword is a logical operator, and is used to combine conditional statements:

Example : if a > b and c > a: print("Both conditions are True")

Or

The or keyword is a logical operator, and is used to combine conditional statements:

Example : if a > b or a > c: print("At least one of the conditions is True")

Not

The not keyword is a logical operator, and is used to reverse the result of the conditional statement:

Example : if not a > b: print("a is NOT greater than b")



Nested If

You can have if statements inside if statements, this is called nested if statements.

Example :

```
x = 41
```

```
if x > 10:
```

```
    print("Above ten,")
```

```
    if x > 20:
```

```
        print("and also above 20!")
```

```
    else:
```

```
        print("but not above 20.")
```



The pass Statement

if statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error.

Example :

```
a = 33  
b = 200
```

```
if b > a:  
    pass
```



Python For Loops

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.

With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.



Looping Through a List

Print each fruit in a fruit list:

Example :

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```



Looping Through a String

Loop through the letters in the word "banana":

Example :

```
for x in "banana":  
    print(x)
```



The break Statement

With the break statement we can stop the loop before it has looped through all the items:

Example : Exit the loop when x is "banana":

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break
```



The continue Statement

With the continue statement we can stop the current iteration of the loop, and continue with the next:

Example : Do not print banana:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```



The range() Function

To loop through a set of code a specified number of times, we can use the range() function,

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example :

```
for x in range(6):  
    print(x)
```



The range() Function

To loop through a set of code a specified number of times, we can use the range() function,

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example :

```
for x in range(6):  
    print(x)
```



The range() Function

Note that range(6) is not the values of 0 to 6, but the values 0 to 5.

The range() function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range(2, 6), which means values from 2 to 6 (but not including 6).

Examples :

```
for x in range(2, 6):  
    print(x)
```



The range() Function

The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, 3).

Examples:

```
for x in range(2, 30, 3):  
    print(x)
```



Else in For Loop

The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

Example : Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):  
    print(x)  
else:  
    print("Finally finished!")
```



Nested Loops

A nested loop is a loop inside a loop.

The "inner loop" will be executed one time for each iteration of the "outer loop".

Example:

```
adj = ["red", "big", "tasty"]  
fruits = ["apple", "banana", "cherry"]
```

```
for x in adj:  
    for y in fruits:  
        print(x, y)
```



The pass Statement

for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

Example:

```
for x in [0, 1, 2]:  
    pass
```



Python While Loops

With the while loop we can execute a set of statements as long as a condition is true.

[break – Continue – Else]

Example: Print i as long as i is less than 6:

```
i = 1
while i < 6:
    print(i)
    i += 1
```



Python Functions

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

In Python a function is defined using the `def` keyword:



Calling a Function

To call a function, use the function name followed by parenthesis:

Example :

```
def my_function():  
    print("Hello from a function")
```

```
my_function()
```



Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

Example :

```
def my_function(fname):  
    print(fname + " Refsnes")
```

```
my_function("Emil")
```



Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

Example :

```
def my_function(fname):  
    print(fname + " Refsnes")
```

```
my_function("Emil")
```



Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

Example:

```
def my_function(fname, lname):  
    print(fname + " " + lname)
```

```
my_function("Emil", "Refsnes")
```



Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a tuple of arguments, and can access the items accordingly:

Example:

```
def my_function(*kids):  
    print("The youngest child is " + kids[2])
```

```
my_function("Emil", "Tobias", "Linus")
```



Keyword Arguments

You can also send arguments with the key = value syntax.

This way the order of the arguments does not matter.

Example:

```
def my_function(child3, child2, child1):  
    print("The youngest child is " + child3)
```

```
my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```



Default Parameter Value

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the default value.

Example:

```
def my_function(country = "Norway"):  
    print("I am from " + country)
```

```
my_function("Sweden")
```

```
my_function("India")
```

```
my_function()
```



Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

Example:

```
def my_function(food):  
    for x in food:  
        print(x)
```

```
fruits = ["apple", "banana", "cherry"]
```

```
my_function(fruits)
```



Return Values

To let a function return a value, use the return statement:

Example:

```
def my_function(x):  
    return 5 * x
```

```
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```



Recursion

Python also accepts function recursion, which means a defined function can call itself.

Example:

```
def tri_recursion(k):  
    if(k > 0):  
        result = k + tri_recursion(k - 1)  
        print(result)  
    else:  
        result = 0  
    return result  
print("\n\nRecursion Example Results")  
tri_recursion(6)
```



Python Modules

Consider a module to be the same as a code library.

A file containing a set of functions you want to include in your application.

To create a module just save the code you want in a file with the file extension `.py`:



Example

Save this code in a file named **mymodule.py**

```
def greeting(name):  
    print("Hello, " + name)
```

Now we can use the module we just created, by using the import statement:

```
import mymodule
```

```
mymodule.greeting("Jonathan")
```



Variables in Module

Save this code in the file **mymodule.py**

```
person1 = {  
    "name": "John",  
    "age": 36,  
    "country": "Norway"  
}
```

Import the module named **mymodule**, and access the person1 dictionary:

```
import mymodule  
a = mymodule.person1["age"]  
print(a)
```



Re-naming a Module

You can create an alias when you import a module, by using the `as` keyword:

Example:

```
import mymodule as mx
```

```
a = mx.person1["age"]  
print(a)
```



Built-in Modules

There are several built-in modules in Python, which you can import whenever you like.

Example:

```
import platform
```

```
x = platform.system()  
print(x)
```



Import From Module

The module named mymodule has one function and one dictionary:

```
def greeting(name):  
    print("Hello, " + name)
```

```
Person1 = { "name": "John", "age": 36, "country": "Norway" }
```

Import only the person1 dictionary from the module:

```
from mymodule import person1
```

```
print (person1["age"])
```



CHECKING FOR UNDERSTANDING

At the end of the training course, check the audience's understanding of the topics discussed.



Speaker. Majid Saqr



THANK YOU FOR LISTENING!

Reach out for any questions.



WEBSITE
expotech.online

EMAIL ADDRESS
contact@expotech.online

PHONE
[01144253820](tel:01144253820)



Speaker. Majid Saqr

